

break and continue Statements in Java

Both `break` and `continue` are control flow statements that alter the flow of execution in loops. They are used to control the loop's behavior based on certain conditions.

`break` Statement

The `break` statement is used to immediately **exit** from a loop or switch statement, regardless of the loop's condition. When the `break` is encountered, the program control moves to the next statement after the loop or switch block.

Syntax for `break` :

```
break;
```

Key Points about `break` :

- It immediately terminates the innermost loop or switch.
- Once the `break` is executed, the program control moves to the first statement after the loop.
- Commonly used to exit loops when a certain condition is met.

Example using `break` :

```
public class BreakExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {  
            if (i == 5) {  
                break; // Exit the loop when i is 5  
            }  
            System.out.println("i = " + i);  
        }  
        System.out.println("Loop ended.");  
    }  
}
```

Output:

```
i = 1  
i = 2
```

```
i = 3
i = 4
Loop ended.
```

- In this example, the loop terminates when `i` is 5 due to the `break` statement. The program control then moves to the line after the loop.

`continue` Statement

The `continue` statement is used to **skip** the current iteration of a loop and proceed with the next iteration. When `continue` is encountered, the rest of the loop body is skipped for that iteration, and the loop control moves to the next iteration.

Syntax for `continue` :

```
continue;
```

Key Points about `continue` :

- It skips the current iteration and moves to the next one.
- The condition (if present) is checked again to determine whether the loop should continue.
- It can be used to skip specific conditions inside loops (e.g., avoid executing certain code for specific values).

Example using `continue` :

```
public class ContinueExample {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i == 5) {
                continue; // Skip the rest of the loop when i is 5
            }
            System.out.println("i = " + i);
        }
    }
}
```

Output:

```
i = 1  
i = 2  
i = 3  
i = 4  
i = 6  
i = 7  
i = 8  
i = 9  
i = 10
```

- In this example, when `i` is 5, the `continue` statement skips the `System.out.println` statement and moves to the next iteration of the loop.

Comparison of `break` and `continue` :

Feature	<code>break</code> Statement	<code>continue</code> Statement
Purpose	Exits the loop or switch immediately.	Skips the current iteration and moves to the next one.
Effect on Loop	Terminates the loop entirely.	Skips the remainder of the current iteration and proceeds to the next iteration.
Use Case	When a condition is met, you want to stop the loop completely.	When you want to skip specific iterations (e.g., avoid processing certain values).
Loop Behavior	The program control moves to the first statement after the loop.	The program control jumps to the next iteration of the loop.
